

SmartKhata Interview Preparation

Candidate Experience: 1.6 years

Project Type: Retail / credit ledger / inventory / staff management web application

Prepared From: Current SmartKhata codebase analysis

Use this document to explain your real project work confidently in interviews. The answers are written in clear English so you can speak naturally.

React

Vite

Node.js

Express

MySQL

Axios

JWT

Recharts

1. Project Summary

SmartKhata is a business management web application built for small shops and retail businesses. It helps manage daily credit, payments, customer ledgers, supplier purchases, inventory, shopping orders, and staff records.

The main goal of this project is to provide a single dashboard where a shop owner can track customer outstanding balances, payment recovery, product stock, order status, supplier debit, and staff attendance/salary data.

2. Best Self Introduction For This Project

"My current project is SmartKhata. It is a full-stack web application where I worked on React frontend, Node.js Express backend, and MySQL database. The business domain is retail credit ledger and inventory management. The project includes customer management, customer product entries, payments with balance updates, supplier purchase tracking, product inventory, shopping orders with stock deduction, dashboard analytics, authentication, and a staff HRMS module. On the frontend, I worked on reusable pages, forms, filters, dashboard charts, API integration using Axios, and responsive UI. On the backend, I implemented REST APIs, validations, MySQL CRUD operations, transaction handling, schema creation, balance calculations, and error handling."

3. Tech Stack

Layer	Technology	Where It Is Used
Frontend	React 19, Vite	Dashboard, Customers, Payments, Inventory, Shopping, Orders, Staff, Login/Register pages
Routing	React Router DOM	Page navigation: /dashboard, /customers, /payment, /staff, /orders, etc.
API Calls	Axios	Service files such as customerService, paymentService, staffService, shoppingService
Charts/UI	Recharts, React Icons	Dashboard area chart, cards, icons, HRMS UI
Backend	Node.js, Express	REST APIs under /api/auth, /api/customers, /api/payments, /api/staff, /api/shopping, etc.
Database	MySQL using mysql2	customers, payments, products, stock_movements, shopping_orders, staff, supplier_purchases

Security	bcryptjs, jsonwebtoken	Password hashing and login token generation
----------	---------------------------	---

4. What I Worked On

Frontend Work

- Created React pages such as Dashboard, Customers, Add Customer, Customer Details, Payment, Inventory, Shopping, Orders, Supplier, Staff, Reports, and Settings.
- Configured page navigation using React Router.
- Implemented a sidebar-based application layout.
- Created Axios service files for customer, payment, staff, shopping, inventory, and authentication APIs.
- Built dashboard views for customer count, total credit, total payment, pending amount, recent transactions, low stock alerts, and charts.
- Built an HRMS-style Staff module with registration, filters, staff list, profile, attendance entry/history, salary preview, document section, and report actions.
- Added client-side form validations for mobile number, email, salary, and required fields.

Backend Work

- Set up the Express server with CORS and JSON middleware.
- Created module-wise REST routes and controllers.
- Implemented APIs for customer CRUD, customer history, and customer product entries.
- Used MySQL transactions in payment APIs so customer current balance updates correctly during payment add/update/delete operations.
- Implemented product inventory APIs including stock status, stock movement history, and add/update/delete product logic.
- Implemented shopping order APIs with order creation, order item insertion, stock deduction, and stock restore logic on order cancellation.
- Implemented staff APIs for staff CRUD, summary, attendance, salary payment, advance payment, and history.
- Implemented authentication with registration, login, bcrypt password hashing, and JWT token generation.

Database Work

- Used MySQL as the database. The current database configured in code is **adesh**.
- Main tables include smartkhata_register_data, customers, customer_product_entries, payments, products, stock_movements, shopping_orders, shopping_order_items, suppliers, supplier_purchases, customer_credits, staff, staff_attendance, staff_salary_payments, and staff_advances.
- The project uses CREATE TABLE IF NOT EXISTS and ALTER TABLE based schema setup inside some controllers.
- Important calculations include customer current_balance, payment previous/current balance, supplier pending amount, inventory stock_qty, order line_total, and order total_amount.

5. Module Explanation

Module	Functionality	Important APIs/Tables
Authentication	User registration/login, password hash, JWT token	/api/auth/register, /api/auth/login, smartkhata_register_data

Customer	Add/update/delete customers, product entries, customer history	/api/customers, customers, customer_product_entries
Payments	Payment add/update/delete, customer balance update, payment summary	/api/payments, payments, customers.current_balance
Inventory	Products, stock quantity, low stock, stock history	/api/inventory, products, stock_movements
Orders	Create shopping orders, order items, status update, stock deduction/restore	/api/shopping/orders, shopping_orders, shopping_order_items
Supplier	Supplier records and supplier purchase/debit tracking	/api/suppliers, /api/supplier-purchases, suppliers, supplier_purchases
Staff HRMS	Staff registration, filtering, attendance, salary/advance history	/api/staff, staff, staff_attendance, staff_salary_payments, staff_advances
Dashboard	Business overview, cards, charts, recent transactions, low stock alerts	Consumes customer, credit, payment, order, and inventory APIs

6. Interview Questions And Answers

Q1. Tell me about your SmartKhata project.

SmartKhata is a full-stack business management application for small retailers. It manages credit ledgers, payment recovery, customer records, suppliers, inventory, shopping orders, and staff management. It uses React for the frontend, Express for the backend, and MySQL for the database. I worked on frontend screens, API integration, backend REST APIs, database queries, validations, and balance/stock calculation logic.

Q2. What was your role in the project?

My role was full-stack developer. On the frontend, I worked on React pages, forms, dashboard, filters, reusable services, and API integration. On the backend, I worked on Express routes/controllers, MySQL CRUD operations, authentication, payment transaction handling, inventory stock updates, and staff attendance APIs.

Q3. Why did you choose React for the frontend?

React is component-based, so it is easy to divide a large UI into pages and components. SmartKhata has many modules like Dashboard, Customers, Staff, Payment, and Inventory, so React state management, conditional rendering, and reusable components were useful.

Q4. How does the frontend communicate with the backend?

The frontend calls backend REST APIs using Axios. For example, `customerService.js` calls customer APIs, `paymentService.js` calls payment APIs, and `staffService.js` calls staff APIs. The common backend base URL is mostly `http://localhost:5000/api`.

Q5. Explain your backend architecture.

The backend is based on Node.js and Express. `server.js` registers middleware and routes. Each module has a route file, such as `customerRoutes`, `paymentRoutes`, and `staffRoutes`. The business logic is written in controllers. The database connection is created in `config/db.js` using `mysql2`.

Q6. Which database did you use and why?

I used MySQL because the project has structured relational data such as customers, payments, orders, order items, products, suppliers, and staff. SQL is suitable for relationships, reporting queries, and financial calculations.

Q7. How did you handle payment and customer balance?

When a payment is added, the backend locks the customer row using `SELECT ... FOR UPDATE`, reads the previous balance, subtracts the payment amount, inserts a payment record, and updates the customer `current_balance`. This operation runs inside a MySQL transaction to maintain data consistency.

Q8. Why did you use database transactions?

Payment and order flows contain multiple database operations. If a payment is inserted but the customer balance update fails, data becomes inconsistent. With transactions, all operations are committed together, or if any step fails, everything is rolled back.

Q9. How does inventory stock update work?

The products table maintains `stock_qty`. During product update, the system calculates the difference between old stock and new stock and inserts an ADD or REDUCE record into `stock_movements`. When an order is created, stock is reduced. When an order is cancelled, stock is restored.

Q10. What is the order flow?

The user creates a shopping order with customer name, mobile number, pickup time, and selected products. The backend validates product stock, creates a record in `shopping_orders`, inserts records in `shopping_order_items`, reduces stock, and adds ORDER entries in `stock_movements`.

Q11. How did you implement authentication?

The register API validates user data, checks duplicate email, hashes the password using bcrypt, and saves the user in the database. The login API fetches the user by email, verifies the password using bcrypt.compare, and generates a JWT token on successful login.

Q12. What validations did you add?

On the frontend, I added validations for required fields, mobile number, email, and salary. On the backend, I added validations for positive amount, valid date format, valid IDs, stock availability, paid amount not exceeding total amount, and existence checks for customer, staff, and product.

Q13. Explain the Staff module.

The Staff module is an HRMS-style module. It includes staff registration, list, search, department/status filters, profile view, attendance entry, attendance history, salary dashboard, salary payment/advance backend APIs, and report/export action buttons. The backend uses staff, staff_attendance, staff_salary_payments, and staff_advances tables.

Q14. What does the Dashboard do?

The Dashboard fetches data from multiple APIs: customers, credits, payments, orders, and inventory products. It displays total customers, total credit, total payment, pending balance, recent transactions, top products, low stock alerts, and charts.

Q15. How did you handle API errors?

Backend controllers use a sendError helper to return consistent JSON responses with success false and a message. On the frontend, try/catch blocks display useful error messages, such as staff save failed or backend/database check message.

Q16. What is the difference between PUT, POST, DELETE, and GET in your project?

GET is used to fetch data, POST is used to create a new record, PUT is used to update an existing record, and DELETE is used to remove a record. For example, POST /api/staff adds staff, PUT /api/staff/:id updates staff, DELETE /api/staff/:id deletes staff, and GET /api/staff fetches staff.

Q17. How did you design reusable services?

I separated API logic into frontend service files. Components do not need to directly hardcode every Axios URL. Instead, customerService, paymentService, staffService, shoppingService, and inventoryService are used. This makes the code more maintainable.

Q18. How do you protect against SQL injection?

Most queries use parameterized placeholders like ?. Values are passed separately in an array. This avoids directly concatenating user input into SQL strings and reduces SQL injection risk.

Q19. What challenges did you face?

The main challenge was maintaining balance and stock consistency. During payment add/update/delete, customer balance must remain correct. During order cancellation or reopening, stock must be restored or reduced correctly. I handled this using transactions, row locking, validations, and stock_movements history.

Q20. What improvements are pending?

The Reports page and Settings page are still in placeholder stage. The unified Transactions ledger API is not fully active. Auth middleware and ProtectedRoute need to be applied consistently. Database credentials should be moved to .env. Automated tests should also be added.

Q21. How would you improve this project for production?

I would move database credentials to .env, read the JWT secret from environment variables, apply protected routes, use an input validation library like Joi or Zod, add automated API tests, create proper database migrations, improve logging, and set up deployment.

Q22. What is bcrypt and why did you use it?

bcrypt is a password hashing library. Storing plain passwords in the database is unsafe. During registration, the password is hashed, and during login, the entered password is compared with the stored hash.

Q23. What is JWT?

JWT is a token-based authentication mechanism. After successful login, the backend generates a token. The frontend can store this token and send it in the Authorization header while calling protected APIs.

Q24. What is useEffect used for in your React pages?

useEffect is used to load data when a page or component mounts. For example, the Dashboard calls customers, payments, orders, and products APIs when it loads. The Staff page fetches staff summary and staff list when it loads.

Q25. What is useMemo used for?

useMemo is used to avoid recalculating expensive derived data on every render. In the Dashboard, totals, filtered transactions, top products, and low stock data are derived using memoized calculations. In the Staff page, the filtered/sorted staff list is calculated using useMemo.

Q26. Explain the customer history feature.

Customer history combines payments, customer product entries, and shopping order items. The backend runs separate queries, maps the data into a common format, combines the results, and returns them sorted by date in descending order.

Q27. How do you calculate pending amount in a product entry?

In a product entry, $\text{total_amount} = \text{price} * \text{quantity}$. $\text{pending_amount} = \text{total_amount} - \text{paid_amount}$. If the paid amount is greater than the total amount, the backend returns a validation error.

Q28. How does the supplier purchase module work?

Supplier purchase stores supplier_name, purchase_from, amount, paid_amount, bill_no, purchase_date, product_details, and note. pending_amount is calculated in the query as $\text{amount} - \text{paid_amount}$. The summary API returns total debit, total paid, and total pending.

Q29. What is your strongest contribution?

My strongest contribution is the payment/balance and inventory/order flow because these contain business-critical calculations. I worked on API validation, transaction handling, current_balance update, stock deduction/restore, and history tracking.

Q30. How will you explain 1.6 years of experience honestly?

"I have 1.6 years of hands-on experience in full-stack development. I am comfortable with React frontend, Node Express APIs, MySQL queries, CRUD operations, form validations, API integration, and debugging. I am still learning complex architecture design independently, but I can confidently handle module-level development and issue fixing."

7. Short Technical Revision

- **React:** Components, state, props, useEffect, useMemo, routing, forms.
- **Node/Express:** Middleware, routes, controllers, req/res, status codes.
- **MySQL:** CRUD, joins, transactions, aggregate queries, DATE_FORMAT, SELECT FOR UPDATE.
- **API:** GET/POST/PUT/PATCH/DELETE, JSON response, error handling.
- **Security:** bcrypt hashing, JWT token, environment variables, protected routes.
- **Debugging:** Browser console, network tab, backend logs, database query errors.

8. Questions You Can Ask The Interviewer

- What frontend and backend tech stack does your team use?

- How does your team follow testing and code review processes?
- Do developers get module ownership?
- What is your process for database migrations and deployment?

9. Final Pitch

“Through the SmartKhata project, I learned real business workflows such as credit ledger, payments, inventory, order stock, supplier debit, and staff attendance. I was not limited to UI work only; I also implemented backend APIs, MySQL queries, validations, and business calculation logic. Because of this experience, I can quickly contribute to a junior to mid-level full-stack role.”